

AT.PL: Sheet One

①

If you did + LC, you will be familiar with the derivation game and how it is all about applying the connect and matching rules.

Here is a little recipe for doing these:

1. Remind yourself of the rules
2. Write down what you want to prove
3. (Unk.) you hit axioms and have dealt with all proof obligations:
 - select rule with premise that matches what you wanna prove
 - apply it

Odd/Even Rules:

Because the concepts of odd and even are defined using rules that depend on each other, this is an example of "Simultaneous Generation of Judgements".

EVENZ

zero even

This "axiom" called so as it has no premises, tells us that zero is even

ODD

n even
succ(n) odd

If the predecessor of something is even we know it is odd

EVEN

n odd
succ(n) even

If the predecessor of something is odd we know it is even

zero even EVENZ
succ(zero) odd ODD
succ(succ(zero)) even EVEN
succ(succ(succ(zero))) odd ODD
↳ "3 is odd"

The rule naming here is super helpful because to prove something to be odd, we need to reach for the odd rule.

② i)

I'm a Haskell gal, and Alex already drew connections between R's and Haskell data types, so I'll start with what I know:

```
> data.NatList
> = Empty
> 1 cons Nat NatList
```

$\Rightarrow$ I'll need two rules for lists of nats,
and I'll need to call out to our nat
rules

Let's do the easy one first:

[] natlist EMPTY ← same as constructor

Hasbelle syntax what I've called my judgement / type

Now for the more complicated one. The canister has two drops, so I will need two premises (just like I succeeded one).

$$\frac{n : \text{nat} \quad ns : \text{natlist}}{n : ns : \text{natlist}} \text{CONS}$$

Seems like we can test it by seeing
if we can show $2^{0.5}$ is a root
list:

$\frac{\text{zero nat}}{\text{zero : nat}} \quad \frac{\text{[] natlist}}{\text{[] : natlist}} \quad \text{EMPTY CONS}$

ii)

This will be very similar to the induction rule for natural numbers:

Let P be a property of natural numbers.

If

- $P(\text{zero})$
- whenever $P(n)$, $P(\text{succ}(n))$ holds

then $P(n)$ holds for all n nat.

It seems that an induction principle needs a proof obligation for each rule, and that if a rule has premises we can assume them ("induction hypothesis") to prove the conclusion.

Now let's adjust this for lists:

Let P be a property of lists of natural numbers.

If

- $P([])$ -- empty
- whenever n nat and $P(ns)$, $P(n:ns)$ holds -- cons

[obligation
part is

then $P(ns)$ holds for all ns natlist.

assume
premises

ii) help me already almost did this
 but with a simpler example.

This is the same game as (1) but
 with our rules

$$\begin{array}{c}
 \frac{}{\text{zero nat}}^Z \quad \frac{\frac{}{\text{zero nat}}^Z}{\text{succ}(\text{zero}) \text{ nat}}^S \quad \frac{}{[]}^{\text{empty}} \text{ natlist} \\
 \frac{}{\text{zero nat}}^Z \quad \frac{\text{succ}(\text{zero}) : [] \text{ natlist}}{}^{\text{cons}} \\
 \hline
 \text{zero} : \text{succ}(\text{zero}) : [] \text{ natlist} \\
 \text{"[0, 1]"}
 \end{array}$$

③i) Here we go again!
This is a great way to acquaint yourself
with some rules.

$$\begin{array}{r}
 \text{zero nat} \quad z \\
 \hline
 \text{succ}(\text{zero}) \text{ nat} \quad s \\
 \hline
 \text{sum}(\text{zero}, \text{succ}(\text{zero}), \text{succ}(\text{zero})) \quad \text{BASE} \\
 \hline
 \text{sum}(\text{succ}(\text{zero}), \text{succ}(\text{zero}), \text{succ}(\text{succ}(\text{zero}))) \quad \text{IND} \\
 \hline
 1 + 1 = 2 : 0
 \end{array}$$

ii).

$\text{sum} :: \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{sum } \text{zero } b = b$

-- BASE

$\text{sum } (\text{succ } a) b = \text{succ } (\text{sum } a b)$

-- IND

Yes this uses pattern matching, and you can almost think of that as deciding which rule you apply.

One case per rule. BASE is basically identical but IND requires a recursive call.

iii)

this smells like a proof by induction!
why? Well, so far in the course that is how you have been taught to prove things, this is the judgements/induction sheet, after all, and finally, not on a course context note, the things we are working with are defined using rules that come with induction principles.

The next question, is what do we induct on? The sum judgement seems like the best choice as it encompasses our premise.

Proof by induction on sum .

[Case: BASE]

$$\frac{b \text{ nat}}{\text{sum}(\text{zero}, b, b)}$$

← remember we get to assume the premises

Zero is a nat by the zero rule:

$$\frac{}{\text{zero nat}} \text{Z}$$

b is a nat as given by a premise.

thus all three args to sum are nats

[Case: IND]

BASE

$$\frac{\text{sum}(a, b, c) \text{ IH}}{\text{sum}(\text{succ}(a), b, \text{succ}(c))} \text{IND}$$

IH = a, b, c are nats.

This leaves us with obligations to show that $\text{succ}(a)$ and $\text{succ}(c)$ are nats.

This is done using the succ rule and the fact that a and c are nats from the IH.

□ IND

□

(iv)

Remember $\text{sum}(a, b, c) \sim a + b = c$.
So, intuitively we know there will be a
furthermore, if we see the judgement
 sum as specifying a function, there
better will be a uniquely defined output.

But how to prove this?

We want to show that for any a nat
b nat there is a unique c nat.

When it comes to showing that something
is the case for all types defined by
rules we always reach for our
favourite tool: induction!

But we can't induct on a and b. This
is science! Only change one variable
at a time.

To know how to pick who to induct on,
just look to the rules and see who has
the most interesting stuff done to them

e.g. In sum b remains unchanged,
so inducting on them won't
tell us much.

a on the other hand is where the
action is at!

Proof by induction on a nat.

[Case: zero]

When a = zero, the BASE rule of sum
is triggered:

$$\frac{}{\text{sum}(\underset{a}{\text{zero}}, \underset{b}{b}, \underset{c}{b})}$$

In this case, $c = b$. No need to create any
result, we already have it in the form of
b!

[Case: succ]

This triggers the IND case of sum :

$$\frac{\text{sum}(x, b, c')}{\text{sum}(\text{succ}(x), b, \text{succ}(c'))} \text{IND}$$

IH = there exists c' such that $\text{sum}(x, b, c')$

Using that and the IND rule, we can
conclude that the c we want for
 $a = \text{succ}(x)$ is $\text{succ}(c')$

□ IND

□

(v) Oh gosh we need to prove something again... I wonder how...

INDUCTION!

Since we have all components to talk about sum, we will induct on that.

Proof by induction on sum.

[Case: BASE]

In this case $c = b$, so since b doesn't change $c = c'$

$\vdash \text{BASE}$

[Case: IND]

$$\frac{\text{sum}(x, b, y)}{\text{sum}(\text{succ}(x), b, \text{succ}(y))} \text{ IND}$$

It! = our statement holds for $\text{sum}(x, b, y)$
i.e. If we had $\text{sum}(x, b, y')$ $y = y'$

$$\Rightarrow c = \text{succ}(y) \text{ and } c' = \text{succ}(y')$$

which are equal by transitivity of $=$

$$c = \text{succ}(y) = \text{succ}(y') = c'$$

(vi)

To answer this, we need to think about what it means to be a function: a function is a unique mapping between inputs and outputs
ah ha!

We have shown outputs to be unique.

Then if we wanted a total function we would need each input to produce an output.

From our proofs of existence and uniqueness in (iv) and (v), we can conclude that sum is a total function on $\mathbb{N} \times \mathbb{N}$.

the moral of the story is: if in doubt, proof by induction.

things I noticed:

- People seem most comfortable inducing on $\mathbb{N}$, but that is not always who we wanna induct on. BECAUSE